HernandezLab-WEB

Estructura del proyecto, código y dependencias

Sistema de gestión de laboratorio clínico construido con React + Vite y Supabase como backend (base de datos, autenticación y permisos), reemplazando una arquitectura previa con backend propio en Express/Postgres. La interfaz se construye con la librería propia **anteriority-ui** sobre Material UI, y el estilo visual del sistema (tema verde) se centraliza en **src/styles/theme.js** y **theme.css**.

Campo	Valor
Nombre del paquete	hernandezlab-web
Framework	React 18 + Vite 5
Backend	Supabase (Postgres + Auth + RLS), sin servidor propio
UI Kit	anteriority-ui (sobre @mui/material)
Ruteo	react-router-dom v6
Gestión de estado	Hooks de React (sin Redux/Zustand); Context API para auth y sucursal activa

Este documento describe cómo está organizado el código fuente, qué hace cada carpeta, y el rol de cada dependencia de terceros — en particular **anteriority-ui**, que es la librería de componentes visuales usada en prácticamente todas las pantallas.

1. Dependencias del proyecto

Declaradas en `package.json`. Se dividen en dependencias de producción (lo que corre en el navegador del usuario final) y de desarrollo (herramientas usadas solo durante el desarrollo/build).

1.1 Dependencias de producción

Paquete	Versión	Para qué se usa
react / react-dom	^18.3.1	Librería base de UI. Todo el sistema está construido como componentes de React.
anteriority-ui	^1.0.0	Librería propia de componentes de interfaz (Sidebar, DrawPanel, TableA, Input, Selector, Button, Badge, Frame, Switch, Checks, etc.). Es la pieza que más se reutiliza: casi ninguna pantalla construye un botón o un formulario desde cero, usa estos componentes como base. Internamente envuelve componentes de Material UI.
@mui/material	^5.16.7	Librería de componentes de Google Material Design. anteriority-ui la usa por debajo (ej. TextField, Autocomplete). El proyecto también la usa directamente para ThemeProvider y CssBaseline en App.jsx.
@emotion/react / @emotion/styled	^11.13.x	Motor de estilos (CSS-in-JS) que usa Material UI internamente para aplicar los temas.
@supabase/supabase-js	^2.45.4	Cliente oficial de Supabase. Reemplaza al backend propio: todas las consultas a la base de datos, autenticación de usuarios y lectura/escritura de tablas pasan por este cliente (ver src/lib/supabaseClient.js).
react-router-dom	^6.26.2	Ruteo de páginas (URLs) dentro de la app de una sola página (SPA): /inventario, /compras, /solicitudes, etc.
lucide-react	^0.446.0	Librería de íconos (los íconos de lápiz, basurero, carrito de compras, reloj de historial, etc. que aparecen en los botones de las tablas).
date-fns	^4.4.0	Utilidades para manejo y formateo de fechas.

1.2 Dependencias de desarrollo

Paquete	Para qué se usa
vite / @vitejs/plugin-react	Servidor de desarrollo y empaquetador (build) del proyecto.
eslint + plugins de react	Revisión automática de calidad/errores de código.

Scripts disponibles (desde la carpeta HernandezLab-WEB):

- `npm run dev` — levanta el servidor de desarrollo (Vite).
- `npm run build` — genera la versión de producción en /dist.
- `npm run lint` — corre ESLint sobre todo el código.
- `npm run preview` — sirve localmente el build de producción.

2. anteriority-ui: la librería de componentes

Es la dependencia más usada del proyecto. Vive en `node_modules/anteriority-ui` y se importa componente por componente, por ejemplo:

```
import DrawPanel from 'anteriority-ui/screens/components/DrawPanel/index';
import Input from 'anteriority-ui/screens/components/Input/index';
import Selector from
'anteriority-ui/screens/components/Material-UI/Components/Selector/index';
import TableA from 'anteriority-ui/screens/components/TableA/index';
```

Componentes usados y dónde aparecen:

Componente	Qué es	Dónde se usa en el sistema
Sidebar (ProSidebar)	Menú lateral de navegación con grupos e íconos	DashboardLayout — el layout general de toda la app logueada
DrawPanel	Panel lateral deslizante que contiene un formulario (crear/editar)	Todos los formularios: Usuarios, Sucursales, Proveedores, Inventario, Solicitudes, Compras
Frame	Tarjeta/modal flotante genérico, con opción isModal	ConfirmDeleteModal (confirmar eliminación) y HistorialLotesModal (historial de lotes de inventario)
Input	Campo de texto/número/fecha, envuelve TextField de MUI	Prácticamente todos los formularios del sistema
Selector	Combobox/autocomplete con opciones value/label	Selección de categoría, rol, sucursal, proveedor, ítem de inventario, etc. Requiere un 'id' único por instancia en la misma pantalla.
TableA	Tabla con estados de carga, error, vacío y reintento	Listados: Usuarios, Sucursales, Proveedores, Inventario, Solicitudes, Compras, historial de lotes
Button	Botón con variantes (primary, outline, ghost, danger), loading, ícono	Acciones en toda la app: crear, guardar, cancelar, eliminar, editar
Badge	Etiqueta pequeña con valor (contador, estado)	Totales en cabeceras de listados, aviso de 'Stock bajo', marca de 'Lote actual'
Switch	Interruptor on/off	Campos de estado activo/inactivo (Usuarios, Sucursales)
Checks	Checkbox individual	Selección de exámenes en el formulario de Solicitudes

Convención importante del proyecto: cada `Selector` debe recibir un `id` explícito y único (ej. `id="compra-proveedor"`), porque el id por defecto del componente colisiona cuando hay más de una instancia en la misma pantalla.

3. Estructura de carpetas (src/)

El código fuente sigue una organización **por feature** (por módulo de negocio), con un puñado de carpetas transversales compartidas por todas las features.

```
src/
  App.jsx                  Define las rutas (react-router) y los providers globales
  main.jsx                 Punto de entrada de la aplicación (ReactDOM.render)

  constants/
    routes.js               Rutas de la app (ROUTES.INVENTARIO, ROUTES.COMPRAS, ...)
    texts.js                TODOS los textos visibles de la UI (español), objeto TEXTS

  lib/
    supabaseClient.js        Cliente único de Supabase (URL + anon key desde .env)

  hooks/                     Hooks globales (no pertenecen a una sola feature)
    useAuth.jsx               Sesión, login/logout, datos del empleado logueado
    useSucursalActiva.jsx     Sucursal seleccionada actualmente (persistida en localStorage)

  styles/
    theme.js                  Tema de Material UI (colores, tipografía) — paleta verde
    theme.css                 Variables CSS globales del tema verde

  utils/
    validators.js             Validadores reutilizables (cédula, nombre, fecha, número, etc.)

  components/
    layout/                   DashboardLayout, Sidebar, SucursalSelector, ProtectedRoute, Rol
eGate
    common/                   ConfirmDeleteModal, PlaceholderModule (compartidos entre featur
es)

  features/                   Un módulo de negocio por carpeta
    auth/LoginPage/            Pantalla de inicio de sesión
    dashboard/                 Pantalla de inicio
    usuarios/                  CRUD de empleados del sistema
    sucursales/                CRUD de sucursales del laboratorio
    proveedores/                CRUD de proveedores
    inventario/                 Catálogo de insumos/reactivos/mobiliario/maquinaria,
                                 + historial de lotes (HistorialLotesModal)
    compras/                    Registro de compras a proveedores (alimenta inventario y lote
s)
    solicitudes/                 Registro de exámenes solicitados por pacientes
```

4. Anatomía típica de una feature

Todas las features siguen el mismo patrón, lo que hace que una vez entendida una, las demás se leen igual. Usando **compras** como ejemplo:

Archivo	Responsabilidad
features/compras/index.jsx	Componente de página. Orquesta los hooks, maneja el estado de abrir/cerrar modales, y arma el layout con Header + Tabla + Modal de formulario.
features/compras/hooks/useCompras.js	Toda la comunicación con Supabase para esta feature: fetchCompras, createCompra. Devuelve estado (lista, loading, error) y funciones de mutación.
features/compras/components/ComprasHeader/	Título de la página + contador total + botón 'Registrar compra'.
features/compras/components/ComprasTable/	Definición de columnas de la tabla (usa TableA de anteriority-ui).
features/compras/components/CompraFormModal/	Formulario de alta (usa DrawPanel). Contiene la validación de campos.
features/compras/components/ItemsCompraList/	Subcomponente del formulario: lista dinámica de líneas de ítems comprados (agregar/quitar filas).

Cada componente visual (.jsx) tiene a su lado un archivo `index.module.css` con sus estilos (CSS Modules): no se usan estilos en línea (`style={{...}}`) en el proyecto.

Flujo de datos típico dentro de una feature

- El hook (ej. `useCompras`) hace las consultas a Supabase y mantiene el estado en memoria con `useState`.
- La página (`index.jsx`) llama al hook y pasa los datos como props a Header, Tabla y Modal.
- El modal de formulario valida los campos (con utils/validators.js) y, si están correctos, llama a la función de creación/edición del hook.
- El hook actualiza Supabase y luego el estado local, lo que re-renderiza la tabla automáticamente — sin necesidad de recargar la página.

5. Backend: Supabase (sin servidor propio)

El proyecto no tiene un backend propio (no hay carpeta /server ni API REST propia). Toda la lógica de datos vive en Supabase:

- **Base de datos Postgres**, con el esquema definido en `SQL/schema.sql` (tablas, relaciones, triggers y funciones).
- **Autenticación** (Supabase Auth) — login por email/contraseña, usado por `src/hooks/useAuth.jsx`.
- **RLS (Row Level Security)** — reglas a nivel de base de datos que determinan qué fila puede ver o modificar cada usuario según su rol (administrativo / recepción) y su sucursal.
- **Triggers SQL** — por ejemplo, al insertar una línea de compra (`detalle_compra`), un trigger aumenta el stock del ítem, crea un registro en `lotes_inventario` y recalcula cuál es el lote 'actual' (el de vencimiento más próximo).

El front se conecta a Supabase a través de un único cliente (`src/lib/supabaseClient.js`), configurado con las variables de entorno `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY` (archivo `.env`, no versionado en git).

Tablas principales (SQL/schema.sql)

Tabla	Para qué
sucursales	Sedes del laboratorio
empleados	Usuarios del sistema (login), con rol y sucursal
pacientes	Pacientes atendidos
examenes	Catálogo de exámenes de laboratorio y su precio
proveedores	Proveedores de insumos/reactivos
inventario	Ítems de stock (insumos, reactivos, mobiliario, maquinaria) por sucursal
lotes_inventario	Historial de lotes recibidos por cada ítem (número de lote, vencimiento, cantidad), usado para saber cuál es el lote 'actual' por vencer primero
reactivos_por_examen	Qué insumos/reactivos consume cada examen
compras_reactivos / detalle_compra	Compras a proveedores y sus líneas de ítems
solicitudes / detalle_solicitud	Exámenes solicitados por un paciente y su estado

6. Resumen visual de capas

Capa	Tecnología	Ejemplo de archivo
Páginas / Rutas	react-router-dom	App.jsx, src/features/*/index.jsx
Componentes de UI	anteriority-ui + @mui/material	Input, Selector, TableA, DrawPanel
Estilos	CSS Modules + tema propio	*.module.css, styles/theme.js/.css
Estado y datos	Hooks de React + @supabase/supabase-js	useCompras.js, useInventario.js
Validación	Utilidades propias	utils/validators.js
Textos	Diccionario centralizado	constants/texts.js (TEXTS)
Backend	Supabase (Postgres + Auth + RLS)	SQL/schema.sql

En síntesis: **anteriority-ui** resuelve toda la parte visual reutilizable (inputs, tablas, paneles, botones), **@mui/material** es la base sobre la que esa librería está construida, y **@supabase/supabase-js** reemplaza por completo la necesidad de un backend propio, comunicándose directamente con Postgres respetando las reglas de seguridad (RLS) definidas en el esquema SQL.

7. Registro de cambios (2026-10-29)

Sesión de trabajo que agregó la pantalla de **Compras**, el **historial de lotes** en Inventario, y corrigió huecos de aislamiento de datos por sucursal. Se documenta aquí porque modifica varias de las piezas descritas en las secciones anteriores.

7.1 Nueva feature: Compras

- Pantalla `src/features/compras/`: registra una compra a un proveedor con número de factura, fecha, y una o más líneas de ítems (cantidad, costo unitario, número de lote, fecha de vencimiento).
- Cada línea de compra, al guardarse, alimenta una nueva tabla `lotes_inventario` — el historial completo de lotes recibidos por cada ítem — y recalcula cuál es el 'lote actual' del ítem: el que tiene cantidad restante y la fecha de vencimiento más próxima (lógica FEFO, *first-expire-first-out*).
- En Inventario se agregó un botón de historial (ícono de reloj) que abre `HistorialLotesModal`, listando todos los lotes de un ítem con el actual resaltado.
- El proveedor ya no se fija al crear un ítem de inventario (se quitó ese campo del formulario): ahora se captura en cada compra, y la tabla de Inventario muestra el proveedor del lote vigente.

7.2 Validación específica por campo

El formulario de Compras pasó de un solo mensaje genérico de error a validar cada línea de forma independiente, señalando exactamente qué falta y en qué línea (ej. "Línea 2: la cantidad debe ser un número mayor a 0"). También se bloquean fechas inválidas: la fecha de compra no puede ser futura, y la fecha de vencimiento de un lote no puede ser anterior a hoy ni anterior a la fecha de la compra.

7.3 Aislamiento de datos por sucursal

El sistema tiene un selector de 'sucursal activa' en la barra superior que debe acotar qué datos ve cada pantalla. Se auditaron todas las features y se encontraron dos huecos reales, ya corregidos:

Feature	Problema encontrado	Corrección
Usuarios	Listaba empleados de todas las sucursales sin importar cuál estuviera seleccionada en el topbar — el selector no tenía ningún efecto en esta pantalla.	useUsuarios ahora recibe idSucursalActiva y filtra con .eq('id_sucursal', idSucursal), igual que Inventario.
Compras	compras_reactivos no tenía columna id_sucursal: una compra no quedaba asociada explícitamente a ninguna sede, solo de forma indirecta a través de a qué ítem de inventario apuntaba cada línea.	Se agregó la columna id_sucursal (migración 002, que rellena las compras existentes a partir de sus líneas) y el trigger de aplicar_detalle_compra ahora rechaza una línea si el ítem comprado pertenece a otra sucursal distinta de la de la compra.

Features revisadas que **no** necesitan este filtro (por diseño, son catálogos compartidos entre sucursales): Proveedores, Exámenes y la propia lista de Sucursales.

7.4 Reconciliación con cambios en paralelo

Mientras se desarrollaba lo anterior, otra sesión de trabajo había subido en paralelo: un resumen funcional en el Dashboard, stock máximo por ítem con aviso de sobre-stock, verificación de código/cédula duplicados antes

de guardar (useDuplicateCheck), badges de 'por vencer'/'vencido' en Inventario, un módulo nuevo de Precios, y el sidebar fijo al hacer scroll. Al integrar ambos trabajos se encontró una colisión real de diseño:

- El otro cambio había agregado un atajo 'Registrar compra' directo desde Inventario (StockMovimientoModal), que sumaba stock y sobrescribía lote/vencimiento del ítem **sin pasar por lotes_inventario ni por una compra formal con proveedor**.
- Se decidió eliminar esa opción de compra rápida (dejando solo 'Sacar stock' para salidas/mermas), porque dejar ambos caminos vivos habría permitido que el historial de lotes quedara incompleto según qué camino usara cada usuario.
- El resto de los cambios en paralelo (stock máximo, duplicados, badges de vencimiento, Precios) se conservaron íntegros y conviven con la nueva lógica de Compras.